Seminar Report

On

A COMPARATIVE SURVEY OF GENERATIVE AI MODELS IN AUTOMATED CODE GENERATION AND SOFTWARE TESTING: TOWARDS ENHANCED SECURITY AND EFFICACY

By

Rishabh Zambre
PRN No.: 1032233565

Under the guidance of

Prof. Shridevi Karande

Dr. Vishwanath Karad MIT World Peace University
School of Computer Science & Engineering
Department of Computer Engineering & Technology
*** 2025-2026 ***

CERTIFICATE

This is to certify that Mr. **Rishabh Zambre** of T.Y. B.Tech., School of Computer Engineering & Technology, Department of Computer Engineering & Technology, CSE-Core, Semester – VI, PRN. No. **1032233565**, has successfully completed seminar on

"A Comparative Survey of Generative AI Models in Automated Code Generation and Software Testing: Towards Enhanced Security and Efficacy"

To my satisfaction and submitted the same during the academic year 2025 - 2026 towards the partial fulfillment of degree of Bachelor of Technology in School of Computer Engineering & Technology under Dr. Vishwanath Karad MIT- World Peace University, Pune.

Prof. Shridevi Karande	**Dr. Balaji Patil**
Seminar Guide	Program Director
Name & Sign	Dept. of Computer Engineering & Technology

ACKNOWLEDGEMENT

I would like to express my profound gratitude to my seminar guide, Prof. Shridevi Karande, for their continuous support, patience, and invaluable guidance throughout this research. Their deep knowledge and technical insights were instrumental in shaping the direction of this comparative survey.

I also extend my sincere thanks to Dr. Balaji Patil, Program Director, and the entire Department of Computer Engineering & Technology at Dr. Vishwanath Karad MIT World Peace University for providing the academic infrastructure and resources necessary to complete this work.

Finally, I am deeply thankful to my family and peers for their unwavering encouragement and support during the preparation of this seminar report.

Rishabh Zambre
Roll No: 10
PRN: 1032233565

ABSTRACT

Generative Artificial Intelligence (GenAI) is already repleting the software development lifecycle, by taking over complicated tasks that previously required considerable human bandwidth. One of the most impactful paths for this technology is found in software testing, especially on the automation of unit tests and validation framework instantiation. The rapid advancement in code generation capabilities of Large Language Models (LLMs) exceeds development of validation procedures in parallel. Modern GenAI architectures often embrace a syntactic functionalist approach, maximizing for executables while methodically ignoring fundamental security requirements and secure architectural principles. The current generation of technologies proves a severe lack in the fundamental validation of AI generated code is being fulfilled, this highlights particularly significant vulnerabilities and explosion practices not adhering to zero-trust principles. In this paper, we provide a detailed comparative survey of GenAI models used for automated code generation and software quality assurance. By challenging the validation layers inherent in LLM processing, this research synthesizes state-of-the-art benchmarking data to suggest multi-dimensional methodologies, that fill the existing security void. The goal is that the software architectures generated by AI must be highly efficient and inherently robust against adversarial attacks.

Keywords: Generative AI, Automated Software Testing, Code Validation, Secure Coding, Vulnerability Assessment, Large Language Models.

INDEX

List of Figures

List of Tables

1. INTRODUCTION

Integrating large language models (LLMs) into the software development lifecycle is a pivotal moment in computing engineering. Generative architectures are far from simple autocomplete functions; they perform complex multimodal understanding, modular generation, and legacy refactoring. In the contingent of software testing, manual test suite authoring expends an average of 40% of a development cycle. AI-powered auto-tests directly address and break this bottleneck in testing through the rapid synthesis of end-to-end unit tests, mocked dependencies and boundary condition validations in seconds allowing developers to focus instead on high level architectural logic.

Yet amidst these unprecedented leaps in productivity, a severe operational dichotomy has revealed itself; the cadence at which functional software is generated today vastly outstrips the ability of current security validation protocols to keep up with. Training foundational models is primarily out on huge corpora of public repositories (e.g., GitHub, The Stack) that contain a blend of secure and highly vulnerable patterns. These models do not have internal mechanisms to assess the security logic of the architecture, resulting in optimization for functional compilation only, rather than against exploitability. As a result, deploying AI-generated code often comes with hidden zero-day vulnerabilities that standard static analysis and linting tools cannot catch.

This report aims at a critical examination of the existing literature relating to the effectiveness and potential security risks associated with code generated by LLMs. It examines how state-of-the-art models are being operated, compares their rates of vulnerability injection, and assesses novel remediation techniques. The organization of this report proceeds as follows: Chapter 2 provides a comprehensive literature survey. Chapter 3 outlines the research methodology and design parameters. Chapter 4 details analytical validation mechanisms. Finally, Chapters 5 and 6 present future research directions and conclusions.

2. LITERATURE SURVEY

The academic discourse surrounding LLM-assisted code generation has rapidly evolved from evaluating functional accuracy to forensically analyzing security implications.

2.1 Vulnerability Injection and Architectural Flaws

Systematic literature reviews over the past few years have confirmed that although LLMs drive optimizations in development velocity, they also consistently copy and paste severe security vulnerabilities throughout [1]. Large-scale datasets on AI-generated code show the recurring themes of poor input sanitization, exploitable cryptographic hashing and insecure access control logic. These results are consistent with research pointing out that the pre-training of foundational models lacks context about zero-trust architecture and therefore defaults to going with the least restrictive functional patterns when given a task versus secure configurations defaulting one [6]. The core issue stems from training data toxicity; reliance on unverified online repositories introduces significant data poisoning risks, leading models to internalize and reproduce deprecated or actively malicious coding paradigms [14].

Table 1: Common Security Vulnerabilities In LLM-Generated Code

Vulnerability Category	Description in GenAI Context	Detection Difficulty	Associated Risk Level
Improper Input Sanitization	Models frequently generate functional data pipelines but omit boundary checks, leading to SQLi or XSS.	High (Often mimics valid logic)	Critical
Insecure Cryptography	AI defaults to deprecated hashing algorithms (e.g., MD5) learned from outdated public repositories.	Medium (Caught by basic SAST)	High
Flawed Access Control	Generates permissive authorization schemas rather than adhering to Zero-Trust Architecture (ZTA).	Very High (Context-dependent)	Critical
Memory Corruption	C/C++ generation often lacks proper buffer allocation/deallocation, causing subtle memory leaks.	High (Bypasses deterministic rules)	High

2.2 The Benchmarking Deficit

This new technology requires novel benchmarking frameworks to evaluate the true security posture of AI-generated code. Conventional security scanners, fine-tuned to human written code, show diminished efficacy in facing LLM output. Vulnerability

2.3 Generative AI in Software Testing

Ironically, the same generative technology that is creating vulnerabilities is being crafted to detect them. Examples of GenAI usage include the generation of multidimensional test data frameworks and culturally appropriate test components, generating automated scenarios that are often omitted by human testers [2]. More advanced implementations use LLMs for exploratory quality assurance and even to do automated penetration testing [12]. Recent work found that LLMs can automatically generate malicious payloads for Cross Site Scripting (XSS) [13] and SQL Injection tests, both complex processes that took considerable time to do manually during vulnerability mapping.

2.4 Remediation and Hardening Strategies

To deal with the security gap we need to take action to prevent problems at both the model and deployment levels. We should test the models in a way that tries to trick them; this is called testing and it is a good way to make sure the models are secure [4]. We should do this when we are fine-tuning the models to stop them from doing things they should not do [8]. At the deployment level using a mix of approaches works best. There are tools like QLPro that combine the ability of Large Language Models to understand context with ways of checking code to find problems [5]. This makes it possible to automatically find vulnerabilities and suggest fixes without needing a person to get involved [5].

3. DETAILS OF DESIGN AND METHODOLOGY

3.1 Search Strategy and Corpus Selection

This review follows the PRISMA guidelines to ensure our method is reliable [6]. We looked at engineering databases like IEEE Xplore and the ACM Digital Library [1]. We also checked preprints from arXiv [9]. We only looked at papers from January 2023 to 2025 [3]. This helps us focus on architectures, not old ones. Our search used keywords like "Generative AI", "Code Generation" and "Vulnerability". We combined them with "LLM", "Automated Program Repair" and "Static Analysis" [19]. This way we found papers on AI code generation and security.

3.2 Inclusion and Exclusion Criteria

To keep the research focused we applied rules to select studies. We only included studies that had data compared different methods and proposed new ways to build systems that deal with the security of code made by machines. These studies had to address how secure the code is and how it is checked [7]. We left out studies that used guidelines to fix

3.3 Data Extraction and Synthesis

We want to know about the computer system that was used, the tools that were used to check if it works, how often problems were put into the system, and how these problems were fixed [4], [8]. We took this information and put it into a chart so we could see what was going wrong with different computer systems [2], [14]. By looking at all of this information we can get an understanding of why the testing of software that uses artificial intelligence is not working very well.

4. ANALYTICAL AND COMPARATIVE VALIDATION WORK

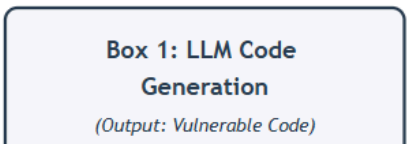
The problem with using Artificial Intelligence to help develop things is that it can make things quickly, but it is hard to know if they are good. To fix this issue we need to look at how we check things and see if it works with the special way that Artificial Intelligence models make things.

4.1 Efficacy of Deterministic SAST and DAST

Traditional Static Application Security Testing and Dynamic Application Security Testing frameworks depend a lot on patterns and predefined rules for how data moves. These rules work well for code written by people. Code written by foundational models is different and often does not follow these rules [7]. When we look at how these models fix problems with memory, we see that the tools we use to check for problems often miss a lot of issues with code that was written by artificial intelligence [16]. This is because these tools have a hard time understanding what the code is supposed to do. They get confused and flag a lot of things that are not really problems [20].

4.2 Synergistic Automated Program Repair (APR)

To fix problems with checking code, new ideas for pipelines suggest combining scanners that find issues with smart frameworks that understand code [5]. These combined frameworks do not use models to create code. Instead, they use them to understand what scanners find. Tests show that using Retrieval-Augmented Generation (RAG) makes fixing vulnerabilities better when the model looks at specific error messages and the code with problems [21]. This approach turns the framework from creating code into an active helper that knows the context. As a result these combinations can fix injection problems and memory leaks without changing how the application works [22].



4.3 Adversarial Vulnerability Seeding

Code validation is getting an approach that involves using an engine to find weaknesses in the code [8]. Instead of just looking for mistakes, special tools are used to create fake attacks on the code. These fake attacks are designed to target the code that the tools made earlier [23]. This process of creating and fixing attacks makes the code stronger. The code is tested again and again with stress tests. This helps make the code more secure against attacks and sneaky data attacks [24].

Table 3: Comparison Of Code Validation Mechanisms Against GenAI Outputs

Validation Mechanism	Operational Paradigm	Efficacy against LLM Code	Key Limitation
Deterministic SAST/DAST	Signature-based scanning and predefined data flow rules.	Low. Misses subtle syntactical variations.	High rate of false negatives; lacks semantic understanding [7], [16].
Synergistic APR (RAG)	Fuses scanners with LLM contextual reasoning.	High. Understands context and suggests secure patches.	Requires significant computational overhead for continuous monitoring [21].
Adversarial Seeding	GenAI generates fake attacks to stress-test generated code.	Very High. Proactively hardens architecture.	Complex to implement and scale across massive legacy codebases [24].

5. FUTURE RESEARCH DIRECTIONS

The focus of code generation needs to change. We need to build security into the code from the start instead of fixing problems after they happen. Researchers should create ways to test how well the code works, how safe it is for memory, and how secure its cryptography is, all at the same time [9]. Another important area for work is Zero Trust Architecture. We should add Zero Trust Architecture rules when we tune the models [6]. If we make Large Language Models assume they are working in a hostile environment, they might avoid creating common security flaws while they write code [17]. This proactive approach could stop authorization and authentication problems before the code even gets added to the project repository [18]. We also need to explore how to make automated repair tools scale better for massive codebases without slowing down the development pipeline [21].

6. CONCLUSION

Generative Artificial Intelligence is completely changing how we build software by

7. REFERENCES

[1] E. Basic and A. Giaretta, "From vulnerabilities to remediation: A systematic literature review of LLMs in code security," arXiv preprint arXiv:2412.15004, 2024.

[2] B. Baudry et al., "Generative AI to generate test data generators," IEEE Software, 2024.

[3] S. C. Dai, J. Xu, and G. Tao, "A comprehensive study of LLM secure code generation," Proceedings of the ACM on Programming Languages, 2025.

[4] J. He and M. Vechev, "Large language models for code: Security hardening and adversarial testing," ACM CCS, 2023.

[5] J. Hu et al., "QLPro: Automated code vulnerability discovery via LLM and static code analysis integration," IEEE Symposium on Security and Privacy, 2025.

[6] L. C. Ramírez et al., "State of the art of the security of code generated by LLMs: A systematic literature review," CONISOFT, 2024.

[7] K. Tamberg and H. Bahsi, "Harnessing large language models for software vulnerability detection: A comprehensive benchmarking study," ICSE, 2024.

[8] X. Yin et al., "Detecting LLM-generated code with subtle modification by adversarial training," USENIX Security Symposium, 2025.

[9] Z. Zheng et al., "A survey of large language models for code: Evolution, benchmarking, and future trends," arXiv preprint arXiv:2311.10372, 2023.

[10] X. Zhou et al., "Large language model for vulnerability detection and repair: Literature review and the road ahead," IEEE Transactions on Reliability, 2024.

[11] M. Cui et al., "Evaluating Large Language Models for Code Generation: A Comparative Study on Python, Java, and Swift," ResearchGate, 2024.

[12] M. Pyhäjärvi, "AI as a tool for exploratory thinking in quality assurance," The Test Pyramid 2.0, 2025.

[13] M. Ćirković et al., "Automated generation of malicious payloads utilizing LLMs in penetration testing," Internet of Things and Cyber-Physical Systems, 2025.

[14] M. A. Ferrag et al., "Generative AI in Cybersecurity: A Comprehensive Review of LLM Applications and Vulnerabilities," Internet of Things and Cyber-Physical Systems, vol. 5, no. 8, 2025.

[15] K. Zhang et al., "CODE AGENT: Enhancing Code Generation with Tool-Integrated Agent Systems," ACL, 2024.

[16] L. Zhang, Q. Zou, A. Singhal, X. Sun, and P. Liu, "Evaluating large language models for real-world vulnerability repair in C/C++ code," Proceedings of the ACM on Programming Languages, pp. 1-10, 2024.

[17] Y. Sheng et al., "A Survey on Large Language Models in Software Security: Opportunities and Threats," MDPI Computers, vol. 15, no. 4, p. 226, 2025.

[18] M. Asare et al., "Security and Quality in LLM-Generated Code: A Multi-Language, Multi-Model Analysis," arXiv preprint arXiv:2502.01853, 2025.